

Lava Architecture

This document collects two architectural surfaces that span multiple modules:

1. **Multi-Tracker SDK** — the layered, pluggable tracker stack added in SP-3a (1.2.0). Pure-Kotlin core, Hilt-injected orchestrator, per-tracker plugin modules, generic primitives in a `basic-digital` submodule.
2. **Local Network Discovery** — the mDNS / NsdManager flow that auto-discovers the lava-api-go service or the legacy `:proxy` on the LAN.

For per-feature MVI shapes, see the relevant `feature/*/README.md` (or the source — every feature mirrors `XxxState + XxxAction + XxxSideEffect + XxxViewModel`).

Multi-Tracker SDK (SP-3a, 1.2.0)

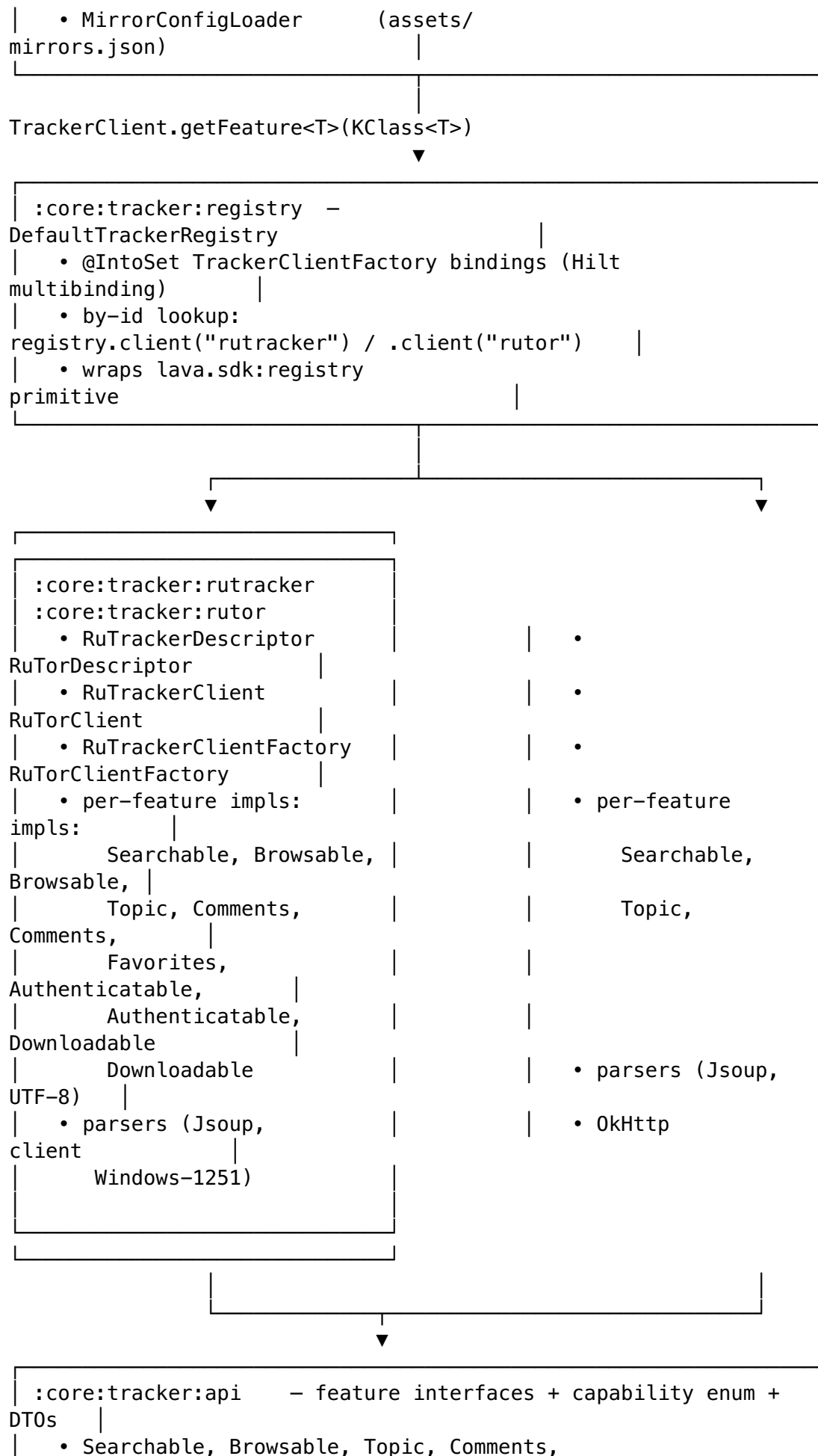
Layering

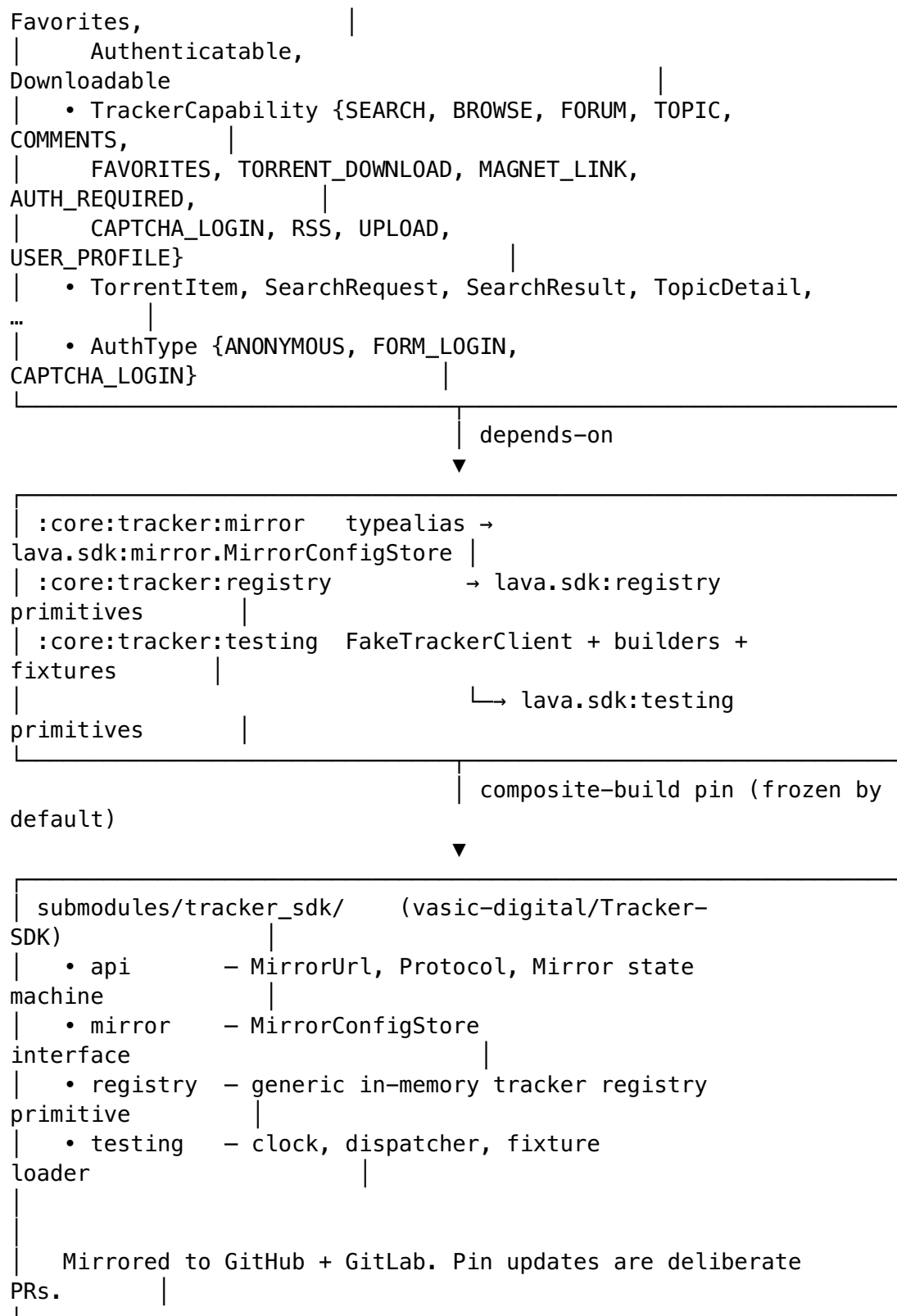
```
| feature:* ViewModels (search_result, topic, login,  
| provider_config, |  
|   credentials_manager, menu) – no per-tracker imports; talk  
| to |  
|   LavaTrackerSdk  
| only |
```

```
| inject @Singleton  
LavaTrackerSdk
```



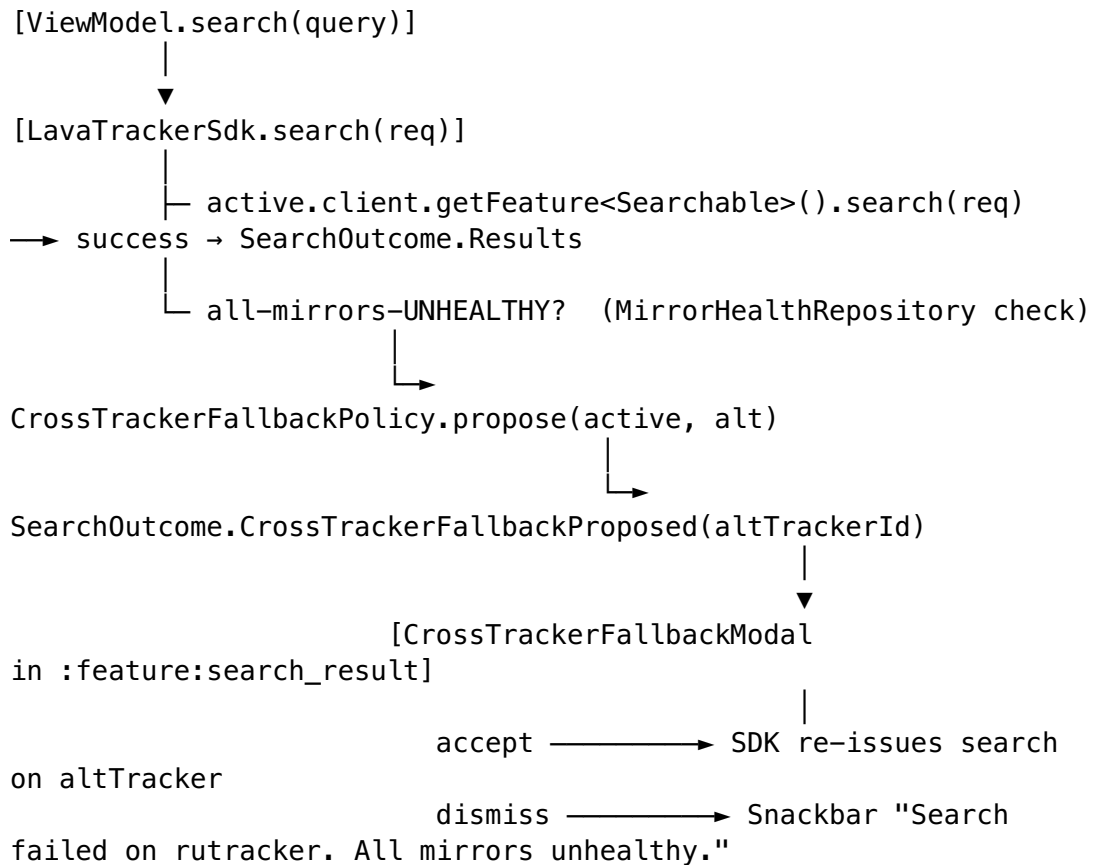
```
| :core:tracker:client – LavaTrackerSdk (Hilt  
| orchestrator) |  
| • activeTracker():  
TrackerClient |  
| • search(req) / browse(...) / topic(...) /  
| download(...) |  
| • CrossTrackerFallbackPolicy →  
CrossTrackerFallbackProposed |  
| • MirrorHealthCheckWorker (15-min  
| PeriodicWork) |  
| • MirrorHealthRepository (Room:  
| tracker_mirror_health) |  
| • UserMirrorRepository (Room:  
| tracker_mirror_user) |
```





The capability enum is the single source of truth. Constitutional clause 6.E (Capability Honesty) requires that **every capability declared in a TrackerDescriptor resolves to a non-null TrackerClient.getFeature<T>()**; the `:core:tracker:registry` test enumerates descriptors + capabilities and asserts non-null at JVM-test time.

Cross-tracker fallback flow



There is **no silent fallback**. The user is always shown the modal and must explicitly accept or dismiss.

Persistence

Room database `AppDatabase v7` owns two SP-3a tables (migration `v6 → v7`):

Table	Entity	Purpose
tracker_mirror_health	MirrorHealthEntity	Per-mirror probe results (HEALTHY/DEGRADED/UNHEALTHY)
tracker_mirror_user	UserMirrorEntity	User-added custom mirrors per tracker

Bundled defaults live at `core/tracker/client/src/main/assets/mirrors.json` and are merged at app start by `MirrorConfigLoader`.

Module reference

Path	Plugin	Purpose
core/tracker/api	lava.kotlin.library + serialization	Public feature interfaces +

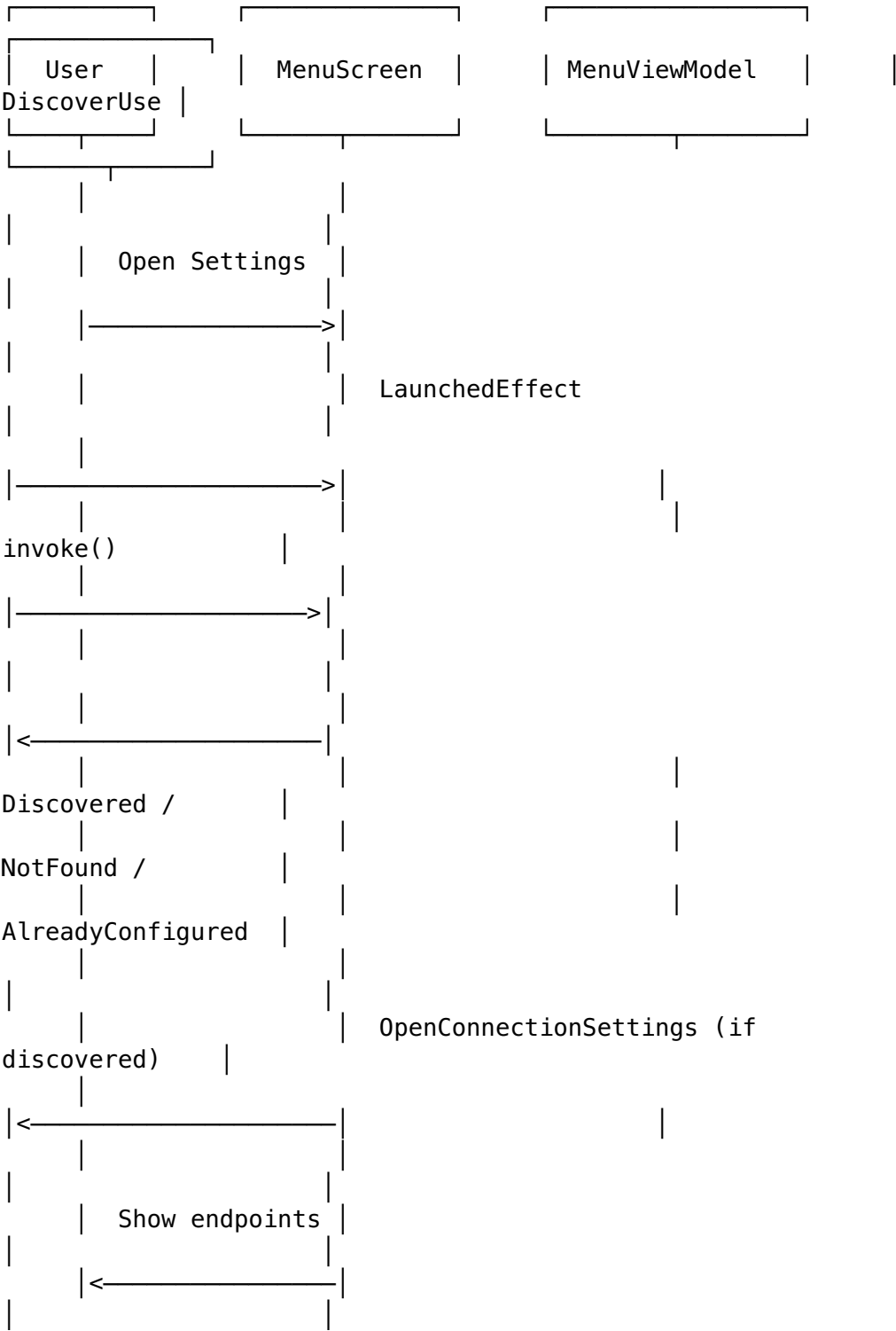
Path	Plugin	Purpose
		capability enum + DTOs
core/tracker/registry	lava.kotlin.library	Lava-domain TrackerRegistry over lava.sdk:registry
core/tracker/mirror	lava.kotlin.library + serialization	MirrorConfigStore typealias bridge
core/tracker/client	lava.android.library + Hilt + serialization	LavaTrackerSdk, persistence repos, WorkManager
core/tracker/rutacker	lava.kotlin.tracker.module	RuTracker plugin
core/tracker/rutor	lava.kotlin.tracker.module	RuTor plugin
core/tracker/testing	lava.kotlin.library	FakeTrackerClient + builders + fixture loaders
feature/provider_config	lava.android.feature + Compose	Per-provider configuration (mirrors, credentials binding, sync toggle, anonymous mode, clone). Reachable from Menu by tapping a provider row. Replaces the SP-3a Trackers screen (deleted in SP-4 Phase C).
feature/credentials_manager	lava.android.feature + Compose	Passphrase-gated credentials CRUD UI (Phase A). Add / edit / delete CredentialsEntry rows.

Generic primitives mounted via composite build at submodules/tracker_sdk/ (submodule pin frozen by default per the Decoupled Reusable Architecture constitutional rule).

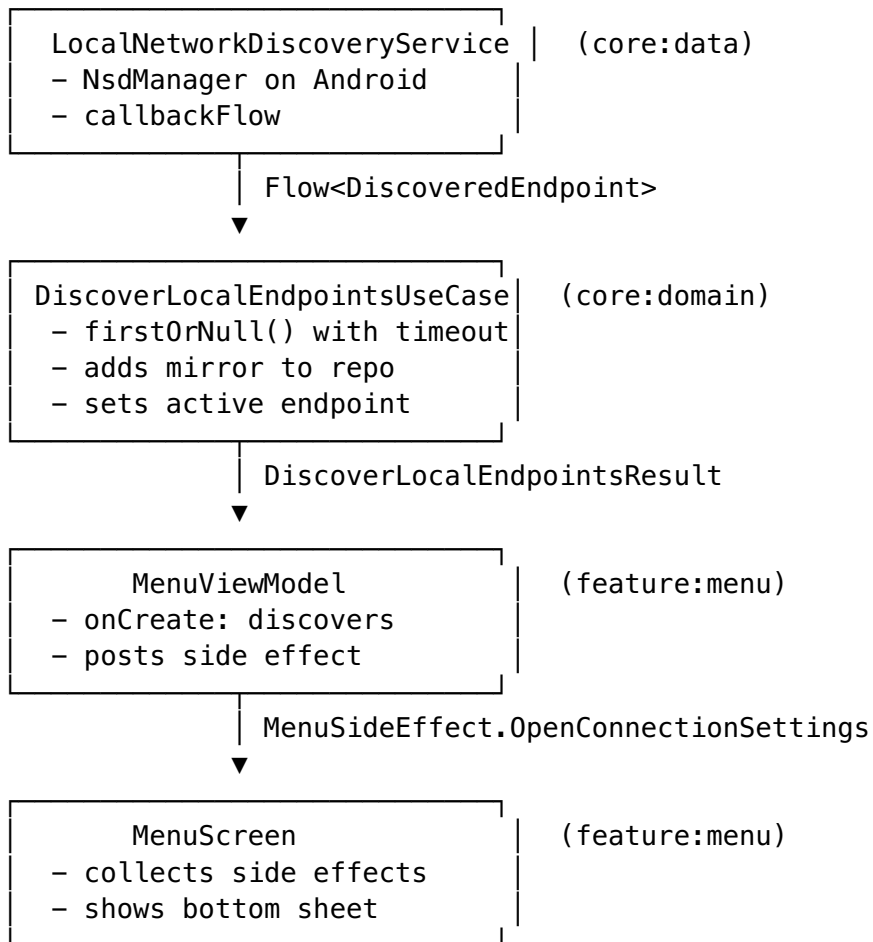
For the full design rationale see [docs/superpowers/specs/2026-04-30-sp3a-multi-tracker-sdk-foundation-design.md](#). For the step-by-step recipe to add a third tracker see [docs/sdk-developer-guide.md](#).

Local Network Discovery Architecture

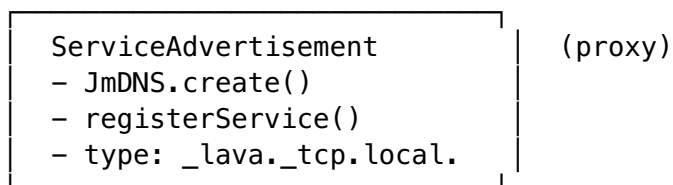
Sequence Diagram



Data Flow



Proxy Advertisement



Module Dependencies

